

NAME: K.Naga Venkata Siva

REGNO: 192425345

1) Register Transfer & Basic ALU Operations Simulator

```

#include <stdio.h>
#include <stdint.h>
#include <string.h>

typedef enum { OP_MOV, OP_ADD, OP_SUB, OP_AND, OP_OR } OpCode;

typedef struct {
    uint8_t R[4]; // Registers R0, R1, R2, R3
} CPUState;

void execute_rtl(CPUState *cpu, OpCode op, int dst, int src1, int src2) {
    uint8_t alu_out = 0;
    switch (op) {
        case OP_MOV: alu_out = cpu->R[src1]; break;
        case OP_ADD: alu_out = cpu->R[src1] + cpu->R[src2]; break;
        case OP_SUB: alu_out = cpu->R[src1] - cpu->R[src2]; break;
        case OP_AND: alu_out = cpu->R[src1] & cpu->R[src2]; break;
        case OP_OR: alu_out = cpu->R[src1] | cpu->R[src2]; break;
    }
    cpu->R[dst] = alu_out; // Transfer result to destination register
    printf("R%d <- %u | State: [R0:%u, R1:%u, R2:%u, R3:%u]\n",
        dst, alu_out, cpu->R[0], cpu->R[1], cpu->R[2], cpu->R[3]);
}

int main() {
    CPUState cpu = {{15, 7, 0, 0}}; // R0=15, R1=7
    printf("Initial: [R0:15, R1:7, R2:0, R3:0]\n");
    execute_rtl(&cpu, OP_ADD, 2, 0, 1); // R2 = R0 + R1
    execute_rtl(&cpu, OP_SUB, 3, 0, 1); // R3 = R0 - R1
    execute_rtl(&cpu, OP_AND, 1, 2, 3); // R1 = R2 & R3
    execute_rtl(&cpu, OP_OR, 0, 1, 2); // R0 = R1 | R2
    return 0;
}

```

OUTPUT:

```

  Console  I/O  AI Agent
Initial: [R0:15, R1:7, R2:0, R3:0]
R2 <- 22 | State: [R0:15, R1:7, R2:22, R3:0]
R3 <- 8  | State: [R0:15, R1:7, R2:22, R3:8]
R1 <- 0  | State: [R0:15, R1:0, R2:22, R3:8]
R0 <- 22 | State: [R0:22, R1:0, R2:22, R3:8]
```

2. Single-Bus vs. Multi-Bus Data Transfer Comparison

```
#include <stdio.h>
```

```
void simulate_transfers() {
    // Task: Perform C = A + B (Registers) and Memory write to I/O
    printf("--- Single-Bus System ---\n");
    printf("Cycle 1: Reg A -> Bus -> ALU Temp Reg 1\n");
    printf("Cycle 2: Reg B -> Bus -> ALU Temp Reg 2\n");
    printf("Cycle 3: ALU Result -> Bus -> Reg C\n");
    printf("Cycle 4: Memory Data -> Bus -> Memory Buffer Register\n");
    printf("Cycle 5: Memory Buffer -> Bus -> I/O Port\n");
    int single_bus_cycles = 5;

    printf("\n--- Three-Bus (Multi-Bus) System ---\n");
    printf("Cycle 1: Bus A (Reg A) + Bus B (Reg B) -> ALU -> Bus C (Reg C) | Bus 3\n");
    printf("Cycle 2: Bus C (Reg C) -> Bus 3 (Mem) -> I/O\n");
    int multi_bus_cycles = 1;

    printf("\nTotal Cycles - Single-Bus: %d, Multi-Bus: %d\n", single_bus_cycles, multi_bus_cycles);
    printf("Speedup: %.2fx\n", (float)single_bus_cycles / multi_bus_cycles);
}

int main() {
    simulate_transfers();
    return 0;
}
```

OUTPUT:

```
Console I/O AI Agent 369 ms ✓

--- Single-Bus System ---
Cycle 1: Reg A -> Bus -> ALU Temp Reg 1
Cycle 2: Reg B -> Bus -> ALU Temp Reg 2
Cycle 3: ALU Result -> Bus -> Reg C
Cycle 4: Memory Data -> Bus -> Memory Buffer Register
Cycle 5: Memory Buffer -> Bus -> I/O Port

--- Three-Bus (Multi-Bus) System ---
Cycle 1: Bus A (Reg A) + Bus B (Reg B) -> ALU -> Bus C (Reg C) | Bus 3 (Mem
-> I/O)

Total Cycles - Single-Bus: 5, Multi-Bus: 1
Speedup: 5.00x
```

3. 5-Stage Instruction Pipeline & Speedup Measurement

```
#include <stdio.h>
```

```
void calculate_pipeline_performance(int num_instructions) {
    int stages = 5;
    // Ideal pipelining: k + (n - 1) cycles
    int pipelined_cycles = stages + (num_instructions - 1);
    // Non-pipelined: k * n cycles
    int non_pipelined_cycles = stages * num_instructions;

    float speedup = (float)non_pipelined_cycles / pipelined_cycles;
    float throughput_pipelined = (float)num_instructions / pipelined_cycles;
    float throughput_non_pipelined = (float)num_instructions / non_pipelined_cycles;

    printf("Instructions: %d\n", num_instructions);
    printf("Pipelined Cycles: %d | Non-Pipelined Cycles: %d\n", pipelined_cycles,
non_pipelined_cycles);
    printf("Speedup: %.2fx\n", speedup);
    printf("Pipelined Throughput: %.3f IPC\n", throughput_pipelined);
    printf("Non-Pipelined Throughput: %.3f IPC\n", throughput_non_pipelined);
}

int main() {
    calculate_pipeline_performance(100);
    return 0;
}
```

OUTPUT:

Console	I/O	AI Agent
---------	-----	----------

```
Instructions: 100
Pipelined Cycles: 104 | Non-Pipelined Cycles: 500
Speedup: 4.81x
Pipelined Throughput: 0.962 IPC
Non-Pipelined Throughput: 0.200 IPC
```

4. Pipeline Hazard Simulation (Stalling, Forwarding, & Branch Prediction)

```
#include <stdio.h>
#include <stdbool.h>

typedef struct {
    int id;
    int src1, src2, dst;
    bool is_branch;
    bool branch_taken;
} Instruction;

void simulate_hazards(Instruction prog[], int count, bool enable_forwarding, bool
branch_predicted_correct) {
    int cycles = 5; // Initial latency
    for (int i = 1; i < count; i++) {
        // Data Hazard Detection (RAW)
        bool raw_hazard = (prog[i].src1 == prog[i-1].dst || prog[i].src2 == prog[i-1].dst);
        if (raw_hazard) {
            if (enable_forwarding) {
                cycles += 1; // 0 stall cycles with forwarding
                printf("Inst %d: RAW Hazard handled via Data Forwarding (0 stalls)\n",
prog[i].id);
            } else {
                cycles += 3; // 2 stall bubbles inserted
                printf("Inst %d: RAW Hazard detected -> 2 Stall Cycles inserted\n", prog[i].id);
            }
        } else {
            cycles += 1;
        }
    }
}
```

```

        // Control Hazard Detection
        if (prog[i-1].is_branch) {
            if (!branch_predicted_correct) {
                cycles += 2; // Pipeline flush penalty
                printf("Inst %d: Branch Misprediction -> 2 Flush Cycles inserted\n",
prog[i-1].id);
            }
        }
    }
    printf("Total Execution Cycles: %d\n\n", cycles);
}

int main() {
    Instruction program[3] = {
        {1, 1, 2, 3, false, false}, // R3 = R1 + R2
        {2, 3, 4, 5, false, false}, // R5 = R3 + R4 (RAW Hazard on R3)
        {3, 5, 0, 0, true, true}   // BEQ R5, 0 (Branch)
    };

    printf("Case 1: No Forwarding, Branch Mispredicted\n");
    simulate_hazards(program, 3, false, false);

    printf("Case 2: Forwarding Enabled, Dynamic Branch Prediction (Correct)\n");
    simulate_hazards(program, 3, true, true);
    return 0;
}

```

OUTPUT:

```

[ ] Console [x] I/O [x] AI Agent
Case 1: No Forwarding, Branch Mispredicted
Inst 2: RAW Hazard detected -> 2 Stall Cycles inserted
Inst 3: RAW Hazard detected -> 2 Stall Cycles inserted
Total Execution Cycles: 11

Case 2: Forwarding Enabled, Dynamic Branch Prediction (Correct)
Inst 2: RAW Hazard handled via Data Forwarding (0 stalls)
Inst 3: RAW Hazard handled via Data Forwarding (0 stalls)
Total Execution Cycles: 7

```

5. 2-Way Superscalar & Simultaneous Multithreading (SMT) Simulation

```
#include <stdio.h>
```

```

typedef struct {
    int thread_id;
    int inst_id;
    int latency; // execution cycles required
} Task;

void run_superscalar_smt(Task queue[], int count) {
    int cycle = 0;
    int completed = 0;
    int idx = 0;

    printf("Cycle | Execution Slot 0 (ALU 0) | Execution Slot 1 (ALU 1)\n");
    printf("-----\n");

    while (idx < count) {
        cycle++;
        Task t0 = queue[idx++];
        if (idx < count) {
            Task t1 = queue[idx++];
            printf("%5d | Thread %d (Inst %d) | Thread %d (Inst %d)\n",
                cycle, t0.thread_id, t0.inst_id, t1.thread_id, t1.inst_id);
            completed += 2;
        } else {
            printf("%5d | Thread %d (Inst %d) | [Idle / Unutilized]\n",
                cycle, t0.thread_id, t0.inst_id);
            completed += 1;
        }
    }
}

printf("\nTotal Cycles: %d | Completed Instructions: %d\n", cycle, completed);
printf("Superscalar IPC (Throughput): %.2f\n", (float)completed / cycle);
}

int main() {
    // SMT Workload combining independent instructions from Thread 0 and Thread 1
    Task workload[6] = {
        {0, 101, 1}, {1, 201, 1},
        {0, 102, 1}, {1, 202, 1},
        {0, 103, 1}, {1, 203, 1}
    };
    run_superscalar_smt(workload, 6);
    return 0;
}

```

OUTPUT:

Console

I/O

AI Agent

387 m

Save

Log

Cycle | Execution Slot 0 (ALU 0) | Execution Slot 1 (ALU 1)

1 | Thread 0 (Inst 101) | Thread 1 (Inst 201)

2 | Thread 0 (Inst 102) | Thread 1 (Inst 202)

3 | Thread 0 (Inst 103) | Thread 1 (Inst 203)

Total Cycles: 3 | Completed Instructions: 6

Superscalar IPC (Throughput): 2.00